



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e
INTERACCIÓN HUMANO COMPUTADORA



REPORTE DE PRÁCTICA N° 02

NOMBRE COMPLETO: PABLO SEBASTIAN NOYOLA
TORRES

N° de Cuenta: 320023213

GRUPO DE LABORATORIO: 03

GRUPO DE TEORÍA: 06

SEMESTRE 2026 - 1

FECHA DE ENTREGA LÍMITE: 01 DE MARZO DE 2026

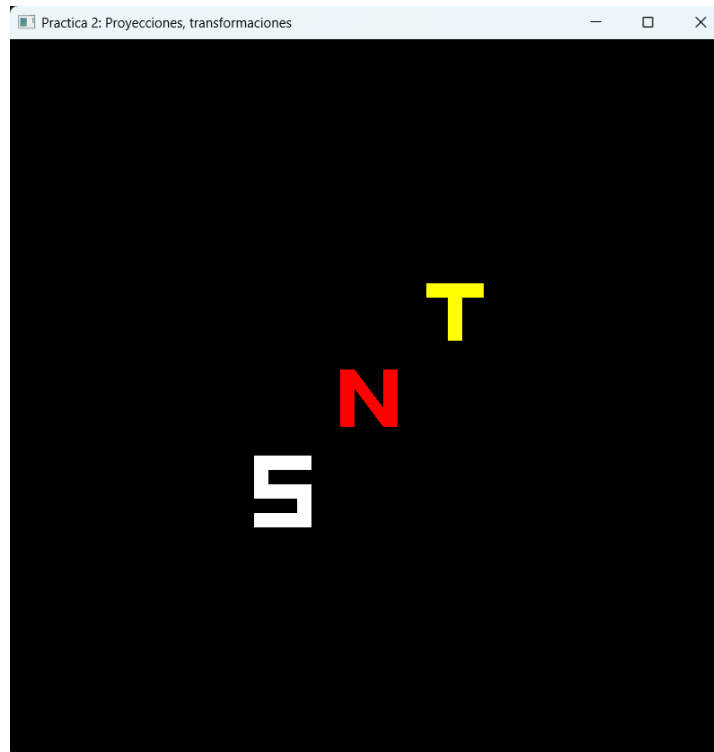
CALIFICACIÓN: _____

REPORTE DE PRÁCTICA:

1.- Ejecución de los ejercicios que se dejaron, comentar cada uno y capturas de pantalla de bloques de código generados y de ejecución del programa.

1.- Dibujar las iniciales de sus nombres, cada letra de un color diferente

Capturas de ejecución:



Código de ejercicio:

```

void CrearLetrasyFiguras()
{
    GLfloat vertices_letras[] = {
        //X      Y      Z      R      G      B
        // Para S (Blanco)
        -0.8f, -0.4f, 0.0f, 1.0f, 1.0f, 1.0f, -0.4f, -0.4f, 0.0f, 1.0f, 1.0f, 1.0f, -0.8f, -0.5f, 0.0f, 1.0f, 1.0f, 1.0f, // Barra sup
        -0.4f, -0.4f, 0.0f, 1.0f, 1.0f, 1.0f, -0.4f, -0.5f, 0.0f, 1.0f, 1.0f, 1.0f, -0.8f, -0.5f, 0.0f, 1.0f, 1.0f, 1.0f,
        -0.8f, -0.5f, 0.0f, 1.0f, 1.0f, 1.0f, -0.7f, -0.5f, 0.0f, 1.0f, 1.0f, 1.0f, -0.8f, -0.7f, 0.0f, 1.0f, 1.0f, 1.0f, // Barra lat
        -0.7f, -0.5f, 0.0f, 1.0f, 1.0f, 1.0f, -0.7f, -0.7f, 0.0f, 1.0f, 1.0f, 1.0f, -0.8f, -0.7f, 0.0f, 1.0f, 1.0f, 1.0f,
        -0.8f, -0.6f, 0.0f, 1.0f, 1.0f, 1.0f, -0.4f, -0.6f, 0.0f, 1.0f, 1.0f, 1.0f, -0.8f, -0.7f, 0.0f, 1.0f, 1.0f, 1.0f, // Barra medi
        -0.4f, -0.6f, 0.0f, 1.0f, 1.0f, 1.0f, -0.4f, -0.7f, 0.0f, 1.0f, 1.0f, 1.0f, -0.8f, -0.7f, 0.0f, 1.0f, 1.0f, 1.0f,
        -0.5f, -0.7f, 0.0f, 1.0f, 1.0f, 1.0f, -0.4f, -0.7f, 0.0f, 1.0f, 1.0f, 1.0f, -0.5f, -0.9f, 0.0f, 1.0f, 1.0f, 1.0f, // Barra lat
        -0.4f, -0.7f, 0.0f, 1.0f, 1.0f, 1.0f, -0.4f, -0.9f, 0.0f, 1.0f, 1.0f, 1.0f, -0.5f, -0.9f, 0.0f, 1.0f, 1.0f, 1.0f,
        -0.8f, -0.8f, 0.0f, 1.0f, 1.0f, 1.0f, -0.4f, -0.8f, 0.0f, 1.0f, 1.0f, 1.0f, -0.8f, -0.9f, 0.0f, 1.0f, 1.0f, 1.0f, // Barra inf
        -0.4f, -0.8f, 0.0f, 1.0f, 1.0f, 1.0f, -0.4f, -0.9f, 0.0f, 1.0f, 1.0f, 1.0f, -0.8f, -0.9f, 0.0f, 1.0f, 1.0f, 1.0f,

        // Para N (Rojo)
        -0.2f, -0.2f, 0.0f, 1.0f, 0.0f, 0.0f, -0.1f, -0.2f, 0.0f, 1.0f, 0.0f, 0.0f, -0.2f, 0.2f, 0.0f, 1.0f, 0.0f, 0.0f, // Poste izq
        -0.1f, -0.2f, 0.0f, 1.0f, 0.0f, 0.0f, -0.1f, 0.2f, 0.0f, 1.0f, 0.0f, 0.0f, -0.2f, 0.2f, 0.0f, 1.0f, 0.0f, 0.0f,
        -0.1f, 0.2f, 0.0f, 1.0f, 0.0f, 0.0f, 0.1f, -0.2f, 0.0f, 1.0f, 0.0f, 0.0f, -0.2f, 0.2f, 0.0f, 1.0f, 0.0f, 0.0f, // Diagonal
        0.1f, -0.2f, 0.0f, 1.0f, 0.0f, 0.0f, 0.2f, -0.2f, 0.0f, 1.0f, 0.0f, 0.0f, -0.1f, 0.2f, 0.0f, 1.0f, 0.0f, 0.0f,
        0.1f, -0.2f, 0.0f, 1.0f, 0.0f, 0.0f, 0.2f, -0.2f, 0.0f, 1.0f, 0.0f, 0.0f, 0.1f, 0.2f, 0.0f, 1.0f, 0.0f, 0.0f, // Poste der
        0.2f, -0.2f, 0.0f, 1.0f, 0.0f, 0.0f, 0.2f, 0.2f, 0.0f, 1.0f, 0.0f, 0.0f, 0.1f, 0.2f, 0.0f, 1.0f, 0.0f, 0.0f,

        // Para T (Amarillo)
        0.4f, 0.8f, 0.0f, 1.0f, 1.0f, 0.0f, 0.8f, 0.8f, 0.0f, 1.0f, 1.0f, 0.0f, 0.4f, 0.7f, 0.0f, 1.0f, 1.0f, 0.0f, // Barra hor
        0.8f, 0.8f, 0.0f, 1.0f, 1.0f, 0.0f, 0.8f, 0.7f, 0.0f, 1.0f, 1.0f, 0.0f, 0.4f, 0.7f, 0.0f, 1.0f, 1.0f, 0.0f,
        0.55f, 0.7f, 0.0f, 1.0f, 1.0f, 0.0f, 0.65f, 0.7f, 0.0f, 1.0f, 1.0f, 0.0f, 0.55f, 0.4f, 0.0f, 1.0f, 1.0f, 0.0f, // Poste ver
        0.65f, 0.7f, 0.0f, 1.0f, 1.0f, 0.0f, 0.65f, 0.4f, 0.0f, 1.0f, 1.0f, 0.0f, 0.55f, 0.4f, 0.0f, 1.0f, 1.0f, 0.0f,
    };
}
MeshColor* letras = new MeshColor();
letras->CreateMeshColor(vertices_letras, 360);
meshColorList.push_back(letras);

```

```

};
MeshColor* letras = new MeshColor();
letras->CreateMeshColor(vertices_letras, 360);
meshColorList.push_back(letras);

//Projection: Matriz de Dimensión 4x4 para indicar si vemos en 2D( orthogonal) o en 3D) perspectiva
glm::mat4 projection = glm::ortho(-5.0f, 5.0f, -5.0f, 5.0f, 0.1f, 100.0f);
//glm::mat4 projection = glm::perspective(glm::radians(70.0f), mainWindow.getBufferWidth() / mainWindow.getBufferHe

//Model: Matriz de Dimensión 4x4 en la cual se almacena la multiplicación de las transformaciones geométricas.
glm::mat4 model(1.0); //fuera del while se usa para inicializar la matriz con una identidad

//Loop mientras no se cierra la ventana
while (!mainWindow.getShouldClose())
{
    //Recibir eventos del usuario
    glfwPollEvents();
    //Limpiar la ventana
    glClearColor(0.0f, 0.0f, 0.0f, 1.0f);
    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT); //Se agrega limpiar el buffer de profundidad

    //Letras-----
    //Para las letras hay que usar el segundo set de shaders con índice 1 en ShaderList
    shaderList[1].useShader();
    uniformModel = shaderList[1].getModelLocation();
    uniformProjection = shaderList[1].getProjectLocation();
    model = glm::mat4(1.0);
    model = glm::translate(model, glm::vec3(0.0f, 0.0f, -2.0f));
    model = glm::scale(model, glm::vec3(2.0f, 2.0f, 2.0f));
    glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
    glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
    meshColorList[0]->RenderMeshColor();
}

```

En este ejercicio, en base a lo realizado en la practica 1, basto con adaptar esas coordenadas de ese código al de esta practica modificando los colores y algunos parámetros.

2.- Generar el dibujo de la casa de la clase, pero en lugar de instanciar triángulos y cuadrados será instanciando pirámides y cubos, para esto se requiere crear shaders diferentes de los colores: rojo, verde, azul, café y verde oscuro en lugar de usar el shader con el color clamp

Capturas de ejecución:



Código de ejercicio:

```
//Vertex Shader
static const char* vShader = "shaders/shader.vert";
static const char* fShader = "shaders/shader.frag";
static const char* vShaderColor = "shaders/shadercolor.vert";
static const char* fShaderColor = "shaders/shadercolor.frag";
//shaders nuevos se crearían acá
static const char* vShaderRojo = "shaders/shaderrojo.vert";
static const char* fShaderRojo = "shaders/shaderrojo.frag";

static const char* vShaderVerde = "shaders/shaderverde.vert";
static const char* fShaderVerde = "shaders/shaderverde.frag";

static const char* vShaderAzul = "shaders/shaderazul.vert";
static const char* fShaderAzul = "shaders/shaderazul.frag";

static const char* vShaderCafe = "shaders/shadercafe.vert";
static const char* fShaderCafe = "shaders/shadercafe.frag";

static const char* vShaderVerdeOscuro = "shaders/shaderverdeoscuro.vert";
static const char* fShaderVerdeOscuro = "shaders/shaderverdeoscuro.frag";
```

```

//Pirámide triangular regular
void CreaPiramide()
{
    unsigned int indices[] = {
        0, 2, 3,
        1, 2, 3,
        0, 2, 4,
        0, 3, 4,
        1, 2, 4,
        1, 3, 4,
    };

    GLfloat vertices[] = {
        -0.5f,  0.0f,  -0.5f,  // 0
        0.5f,   0.0f,   0.5f,  // 1
        0.5f,   0.0f,  -0.5f,  // 2
        -0.5f,  0.0f,   0.5f,  // 3
        0.0f,   1.0f,   0.0f,  // 4
    };

    Mesh* obj1 = new Mesh();
    obj1->CreateMesh(vertices, indices, 16, 24);
    meshList.push_back(obj1);
}

```

```

//Vértices de un cubo
void CrearCubo()
{
    unsigned int cubo_indices[] = {
        // front
        0, 1, 2,
        2, 3, 0,
        // right
        1, 5, 6,
        6, 2, 1,
        // back
        7, 6, 5,
        5, 4, 7,
        // left
        4, 0, 3,
        3, 7, 4,
        // bottom
        4, 5, 1,
        1, 0, 4,
        // top
        3, 2, 6,
        6, 7, 3
    };
};

```

```

    GLfloat cubo_vertices[] = {
        // front
        -0.5f, -0.5f, 0.5f, // <-- Caracteres extraños eliminados
        0.5f, -0.5f, 0.5f,
        0.5f, 0.5f, 0.5f,
        -0.5f, 0.5f, 0.5f,
        // back
        -0.5f, -0.5f, -0.5f,
        0.5f, -0.5f, -0.5f,
        0.5f, 0.5f, -0.5f,
        -0.5f, 0.5f, -0.5f
    };
    Mesh* cubo = new Mesh();
    cubo->CreateMesh(cubo_vertices, cubo_indices, 24, 36);
    meshList.push_back(cubo);
}

```

```

void CreateShaders()
{
    Shader* shader1 = new Shader(); //shader para usar índices: objetos: cubo y pirámide
    shader1->CreateFromFiles(vShaderRojo, fShaderRojo);
    shaderList.push_back(*shader1);

    Shader* shader2 = new Shader(); //shader para usar color como parte del VAO: letras
    shader2->CreateFromFiles(vShaderColor, fShaderColor);
    shaderList.push_back(*shader2);

    Shader* shader3 = new Shader(); //shader para usar índices: objetos: cubo y pirámide
    shader3->CreateFromFiles(vShaderVerde, fShaderVerde);
    shaderList.push_back(*shader3);

    Shader* shader4 = new Shader(); //shader para usar índices: objetos: cubo y pirámide
    shader4->CreateFromFiles(vShaderAzul, fShaderAzul);
    shaderList.push_back(*shader4);

    Shader* shader5 = new Shader(); //shader para usar índices: objetos: cubo y pirámide
    shader5->CreateFromFiles(vShaderCafe, fShaderCafe);
    shaderList.push_back(*shader5);

    Shader* shader6 = new Shader(); //shader para usar índices: objetos: cubo y pirámide
    shader6->CreateFromFiles(vShaderVerdeOscuro, fShaderVerdeOscuro);
    shaderList.push_back(*shader6);
}

```

```

//Casa-----
//Inicializar matriz de dimensi3n 4x4 que servir3 como matriz de modelo para almacenar las transformaciones geom3tricas
// Cubo Rojo
shaderList[0].useShader();
uniformModel = shaderList[0].getModelLocation();
uniformProjection = shaderList[0].getProjectLocation();

model = glm::mat4(1.0);
model = glm::translate(model, glm::vec3(0.0f, 0.0f, -3.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
meshList[1]->RenderMesh();

// Cubo Verde derderecho
shaderList[2].useShader();
uniformModel = shaderList[2].getModelLocation();
uniformProjection = shaderList[2].getProjectLocation();

model = glm::mat4(1.0);
model = glm::translate(model, glm::vec3(0.2f, 0.2f, -2.6f));
model = glm::scale(model, glm::vec3(0.3f, 0.3f, 0.3f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
meshList[1]->RenderMesh();

```

```

// Cubo Verde izquierdo
shaderList[2].useShader();
uniformModel = shaderList[2].getModelLocation();
uniformProjection = shaderList[2].getProjectLocation();

model = glm::mat4(1.0);
model = glm::translate(model, glm::vec3(-0.2f, 0.2f, -2.6f));
model = glm::scale(model, glm::vec3(0.3f, 0.3f, 0.3f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
meshList[1]->RenderMesh();

// Cubo Verde abajo
shaderList[2].useShader();
uniformModel = shaderList[2].getModelLocation();
uniformProjection = shaderList[2].getProjectLocation();

model = glm::mat4(1.0);
model = glm::translate(model, glm::vec3(0.0f, -0.3385f, -2.7f));
model = glm::scale(model, glm::vec3(0.3f, 0.3f, 0.5f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
meshList[1]->RenderMesh();

```

```

// Piramide Azul
shaderList[3].useShader();
uniformModel = shaderList[3].getModelLocation();
uniformProjection = shaderList[3].getProjectLocation();

model = glm::mat4(1.0);
model = glm::translate(model, glm::vec3(0.0f, 0.5f, -3.0f));
model = glm::scale(model, glm::vec3(1.1f, 1.1f, 1.1f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
meshList[0]->RenderMesh();

// Cubo Cafe izquierdo
shaderList[4].useShader();
uniformModel = shaderList[4].getModelLocation();
uniformProjection = shaderList[4].getProjectLocation();

model = glm::mat4(1.0);
model = glm::translate(model, glm::vec3(-1.5f, -0.5f, -3.0f));
model = glm::scale(model, glm::vec3(0.3f, 0.3f, 0.5f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
meshList[1]->RenderMesh();

```

```

// Piramide Verde oscuro izquierdo
shaderList[5].useShader();
uniformModel = shaderList[5].getModelLocation();
uniformProjection = shaderList[5].getProjectLocation();

model = glm::mat4(1.0);
model = glm::translate(model, glm::vec3(-1.5f, -0.4f, -2.95f));
model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
meshList[0]->RenderMesh();

// Cubo Cafe Derecho
shaderList[4].useShader();
uniformModel = shaderList[4].getModelLocation();
uniformProjection = shaderList[4].getProjectLocation();

model = glm::mat4(1.0);
model = glm::translate(model, glm::vec3(1.5f, -0.5f, -3.0f));
model = glm::scale(model, glm::vec3(0.3f, 0.3f, 0.5f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
meshList[1]->RenderMesh();

// Piramide Verde oscuro Derecho

```



```
// Piramide Verde oscuro Derecho
shaderList[5].useShader();
uniformModel = shaderList[5].getModelLocation();
uniformProjection = shaderList[5].getProjectLocation();

model = glm::mat4(1.0);
model = glm::translate(model, glm::vec3(1.5f, -0.4f, -2.95f));
model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
meshList[0]->RenderMesh();
```

Para este ejercicio se modifiqué lo de las figuras en 2d a 3d en sus formas geométricas agregando los archivos de shaders correspondientes para cada figura geométrica.

2.- Liste los problemas que tuvo a la hora de hacer estos ejercicios y si los resolvió explicar cómo fue, en caso de error adjuntar captura de pantalla

Para esta práctica tuve dificultades con el manejo de la librería opengl en algunas cosas que con práctica se mejora y principalmente en la creación de pirámides y cubos.

3.- Conclusión:

Tras la realización de esta segunda práctica considero que fue de gran ayuda para el trabajo con figuras geométricas en 3d, sin embargo aun considero que debo mejorar en mi manejo de opengl.

1. Bibliografía en formato APA

Akenine-Möller, T., Haines, E., & Hoffman, N. (2018). Real-Time Rendering (4th ed.). CRC Press.

Shreiner, D., Sellers, G., Kessenich, J., & Licea-Kane, B. (2013). OpenGL Programming Guide: The Official Guide to Learning OpenGL, Version 4.3 (8th ed.). Addison-Wesley Professional.

Glen-Fiedler, E. (2014). The OpenGL Mathematics Library (GLM). GLM Documentations. <https://glm.g-truc.net/>